SAGE: Self-Assessed Gradient-based Enhancement for Test-Time Alignment of Large Language Models

Anonymous Authors[1]

Abstract

We present SAGE, a test-time alignment method for refining large language model (LLM) outputs at inference time without retraining or relying on external reward models. SAGE uses the same LLM as both generator and self-judge: candidate responses are scored using a calibrated contrastive label judgment prompt and the resulting log-probability margin serves as a continuous preference signal. An iterative refinement algorithm then uses these scores to form good and bad candidate sets, extract a textual gradient, and guide subsequent hypothesis generations under a token budget. Across MATH-500, AlpacaEval, and XSTest, with Qwen3-8B and Llama3-8B-Instruct, SAGE consistently improves accuracy and preference metrics relative to existing test-time alignment approaches, with ablations showing that these gains are robust under fixed inference budgets and do not rely on external reward models. The approach is resource-lean and plug-and-play, and we release the code and prompts to facilitate adoption.

1. Introduction

Large language models (LLMs) pre-trained on vast amounts of unlabeled text have demonstrated strong generalization across a wide range of downstream tasks. Nonetheless, real deployments often require adapting behavior to new domains, user preferences, or safety constraints without the time, data access, or compute needed for retraining. Test-time alignment addresses this need by improving outputs **during inference**, while keeping model weights fixed. It typically allocates a small amount of additional inference compute and elicits interpretable preference signals to steer generation toward more helpful responses aligned with user preferences (Li et al., 2025). This paradigm offers a practical path to domain adaptation and preference alignment under privacy, compliance, and latency constraints, without modifying the pretrained model.

Despite recent progress, existing state-of-the-art test-time alignment approaches, such as TPO (Li et al., 2025) and Args (Khanov et al., 2024), still face several practical limitations. First, many of these methods rely on a separate reward model for scoring, which adds memory and latency overhead and introduces an additional component that can itself be misaligned or exploited. Second, relying solely on the single highest- and lowest-scoring responses for textual critiques can lead to noisy and outlier-sensitive feedback that fails to capture strengths or failure patterns across candidates, resulting in unstable or suboptimal refinement.

We propose SAGE, which treats the base LLM as both a generator and a soft self-judge. Instead of relying on an external reward model, SAGE elicits a calibrated contrastive-label margin from a compact judging prompt, which enables fine-grained, model-internal preference scoring. A key component is an automatic contrastive grouping mechanism that forms representative good and bad sets, yielding a more stable textual 'gradient' than extreme-only (best vs. worst) critiques and reducing sensitivity to outliers. The iterative loop then conditions generation on this gradient to refine future samples. In effect, SAGE leverages the LLM to surface its own implicit reward signal for test-time alignment, avoiding the need for additional reward models while operating under a small inference budget. An overview of SAGE in comparison to common test-time alignment variants is presented in Figure 1.

We conduct experiments on MATH-500 (Hendrycks et al., 2021), AlpacaEval (including length-controlled LC win rate) (Dubois et al., 2024), and XSTest (Röttger et al., 2024), using Qwen3-8B (Yang et al., 2025) and Llama3-8B-Instruct (Grattafiori et al., 2024). These benchmarks cover math reasoning (MATH-500), general instruction following (AlpacaEval), and safety/correctness stress tests (XSTest). We report accuracy on MATH-500/XSTest and win rate and length controlled (LC) win rate for AlpacaEval, comparing against commonly used inference-time baselines, including Best-of-N, TPO (Li et al., 2025) which

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

For ICML 2026 reviewers: When reviewing this paper, please follow the policy for the use of LLMs as specified on your Reviewer Console. Note that the assigned LLM policy might differ from the policy you indicated during reviewer registration.

relies on external reward model, and SPO (Xiang et al., 2025). SAGE consistently improves performance across these benchmarks without relying on an external reward model. On Qwen3-8B, SAGE improves accuracy and preference metrics across all evaluated domains. AlpacaEval win rate vs. GPT-4 increases from 57.8 to 74.9, XSTest accuracy from 89.3 to 93.5, and MATH-500 accuracy from 84.4 to 92.0. SAGE also outperforms inference-time baselines that rely on explicit reward models (e.g., Best-of-N+RM, TPO) and generic LLM-as-a-judge approaches, under comparable budgets. The same procedure also applies to Llama3-8B-Instruct, suggesting that SAGE's self-judged textual-gradient refinement generalizes across model families.

We summarize our contributions as follows:

- We introduce SAGE, a test-time alignment method in which the base LLM provides its own soft reward model via a calibrated contrastive label-margin. As a result, the method avoids external reward models, reducing system complexity and potential reward signal misalignment.
- We design a grouped selection and textual-gradient mechanism that constructs good and bad sets and distills concise guidance from the judge. Compared to best–vs.–worst ranking-based feedback, this leads to more stable refinement by reducing sensitivity to outliers and improving transfer across iterations.
- We provide a practical self-judging recipe, called multi-aspect tagging with weights and confidence-threshold calibration, that turns discrete tags into a calibrated continuous score. This scoring scheme improves hypothesis ranking and provides a reliable correctness signal, particularly for verifiable tasks.
- Finally, we present an iterative test-time alignment procedure that integrates self-judged scoring, grouped selection, and textual gradients to progressively improve generations. We evaluate the resulting method on MATH-500, AlpacaEval (win/LC), and XSTest using Qwen3-8B (Yang et al., 2025) and Llama3-8B-Instruct (Grattafiori et al., 2024). Code, prompts, and scoring configurations are released to support reproducibility, at: `https://anonymous.4open.science/r/SAGE2026-8309`.

2. Related Works

2.1. Preference alignment

Aligning pretrained LLMs with human preferences is often approached by updating model parameters using preference supervision. Existing methods span point-wise optimization, such as policy-gradient variants like PPO (Schulman et al., 2017), ReMax (Li et al., 2024), and KTO (Ethayarajh et al., 2024), which adjust the model using scalar feedback on individual outputs, as well as pairwise optimization approaches that leverage relative judgments. A central strand is DPO (Rafailov et al., 2023), which reformulates RLHF objectives to directly optimize the policy from pairwise comparisons. Subsequent work addresses overfitting and reduces reliance on reference policies, for example by constraining score gaps (Azar et al., 2024), or by removing or adapting the reference model (Meng et al., 2024; Kim et al., 2025; Gorbatovski et al., 2025). These training-time procedures are effective, but they require curated preference data and repeated fine-tuning, which can be slow to iterate and costly to deploy as requirements change.

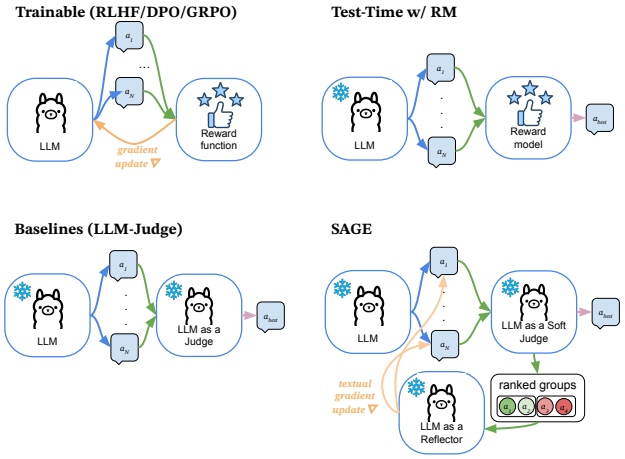


Figure 1. Comparison between test-time alignment methods. SAGE acts post-training and uses the base LLM as generator, soft self-judge with a calibrated contrastive score, and textual-gradient extractor unlike LLM-as-judge heuristics or external RM–based scoring.

2.2. Test-time alignment

Test-time alignment covers decoding heuristics, compute-allocation strategies, and judge-driven selection used at inference without changing model weights. Classical decoding includes greedy and beam search (Reddy, 1976), which track multiple partial hypotheses. Although effective on structured tasks, these methods can over-concentrate probability mass and amplify local errors. Sampling-based decoding encourages diversity. For example, nucleus sampling (Holtzman et al., 2020) truncates low-mass tails, and contrastive search (Su & Collier, 2023) penalizes degeneracy. However, both remain sensitive to hyperparameters.

Beyond fixed heuristics, adaptive search variants add flexibility, for example through alternative normalization schemes or affect-guided steering (Freitag & Al-Onaizan,

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

2017; Huang et al.), but they still rely on hand-tuned policies. A strong and widely used baseline is best-of-n (BoN): draw n candidates and pick the highest under a proxy. When paired with a reward model (RM), BoN+RM can deliver sizable gains via reranking (e.g., human-feedback pipelines such as Stiennon et al., 2020). However, it incurs additional RM training and maintenance costs and risks over-optimization when the proxy is misaligned. Replacing the RM with an LLM judge trades training for prompt engineering. In this setting, judge prompts can be used to verify intermediate steps (e.g., for math (Mahdavi et al., 2025)) or elicit preferences. Test-time preference optimization frameworks (Li et al., 2025) iterate textual feedback to improve candidates. These approaches often rely on discrete labels or scalarized heuristics, which can be coarse, sensitive to verbosity, and inefficient in their use of tokens.

Recent work scales test-time compute for agents and stepwise reasoning by allocating budget across perception, planning, and tool use (Zhu et al., 2025) or by using intermediate checkpoints to guide sampling (Wang et al., 2025). These approaches can improve robustness, but they do so at the cost of increased controller complexity and additional hyperparameters. Efficiency advances for BoN include TreeBoN (Qiu et al., 2025), which introduces speculative tree search together with token-level preference signals (e.g., DPO-style rewards) to guide branch expansion and pruning. This design reduces computational cost while largely preserving output quality.

Prompt-optimization methods aim to improve elicitation rather than search, for example through self-supervised prompt tuning (Xiang et al., 2025). However, these approaches often depend on seed exemplars and can struggle to transfer across domains.

In contrast, SAGE uses the base LLM as both generator and soft judge and replaces brittle discrete votes with a calibrated continuous margin computed directly from a judging prompt. The resulting score supports fine-grained ranking, and a concise textual "gradient" then steers subsequent sampling. This preserves BoN-style simplicity and avoids external reward models and heavy heuristics. The approach also fits strict token budgets through lightweight judging and caching.

3. Proposed Method

3.1. Notation and Problem setup

Let M denote a pretrained (frozen) language model. For a prompt $p \in \mathcal{P}$, the model M induces a conditional policy $\pi_M(h \mid p)$ over responses $h \in \mathcal{X}$. At test time, alignment is typically driven by a scalar preference signal $s(p, h)$, for example from a reward model or an LLM judge, which scores how well h matches human preferences. Our test-time alignment goal is to select a response that maximizes this signal, without updating M:

$$h^\star(p) \;\in\; \arg\max_{h \in \mathcal{X}} s(p, h)\,,$$

Concretely, test-time alignment methods implement a procedure that interacts with M to propose, score, and refine candidates at inference time.

We identify two practical shortcomings of existing test-time alignment approaches: First, many methods rely on a separate reward model for scoring, which adds memory, latency, and maintenance overhead and can introduce proxy mismatch or reward hacking. Second, forming a stable textual 'gradient' for improving prior generations is difficult when methods that compare only the single best and worst candidates (e.g., TPO (Li et al., 2025)), which is outlier-sensitive and often produces noisy guidance that fails to capture broader error patterns across the candidate set. In the next section, we describe our solution to address these limitations.

3.2. SAGE

In this work, we propose SAGE, which uses a single LLM as the generator, the soft judge (reward model), and the textual-gradient estimator. The method assigns soft scores to rollouts and dynamically constructs contrastive groups to obtain a textual gradient, which is then used to further improve generation quality.

At a high level, SAGE uses the base LLM as both generator and soft judge. It draws small batches of candidates, scores them using a calibrated contrastive-label signal, aggregates evidence across groups to extract a concise textual 'gradient', and conditions subsequent sampling on this guidance, all under a fixed token budget. An overview of the method is shown in Figure 2.

3.2.1. Initial generation

At the beginning of the algorithm, we generate an initial set of rollouts (hypotheses) using the LLM as the generator and store them in the rollout set $\mathcal{H}$, which is updated in subsequent iterations. These rollouts are then to be scored by the judge.

3.2.2. Judge

Structured judge prompts and label sets. We leverage the model's implicit preference signal at test time by asking the same LLM to judge its own outputs along a small set of human-interpretable aspects, such as usefulness, verification, completeness, and repetition. For each aspect $k \in \{1, \ldots, K\}$ we build a concise, deterministic judge prompt $P_j^{(k)}(p, h)$ that specifies an aspect-specific rubric and includes the original task prompt p together

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

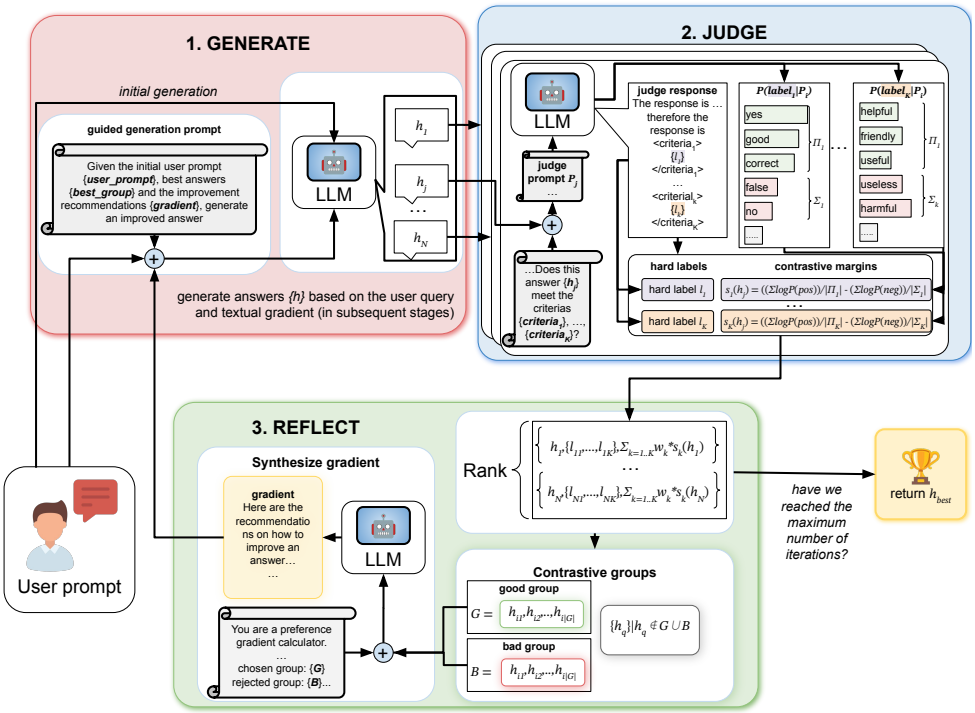


Figure 2. The SAGE framework: The LLM acts as generator, self-judge, and reflector. It extracts a textual gradient ∇ from grouped candidates to iteratively refine the output.

with the candidate response $h \in \mathcal{H}$, and requires the model to emit a decision inside dedicated tags, e.g. `<is_useful>yes|no</is_useful>` (see Appendix Section A). Parsing the tag yields a hard label $\hat{\ell}_k \in \Pi_k \cup \Sigma_k$, where Π_k (resp. Σ_k) is the set of positive (resp. negative) surface forms, e.g. {`yes`} and {`no`}.

From tagged decisions to calibrated margins. Beyond the hard tag, we use the model's token-level probabilities to obtain soft confidence score. Let u_k be the fixed prefix in $P_j^{(k)}(p, h)$ up to, but not including, the first label token. For any $\ell \in \Pi_k \cup \Sigma_k$, we estimate the conditional log-probability $\log P(\ell \mid P_j^{(k)}, u_k)$ by summing the per-token log-probabilities for ℓ; if ℓ is multi-token, we evaluate the full string using teacher forcing and aggregate over its canonical tokenizations via log-sum-exp.[1] This yields a contrastive label margin for each aspect

$$s_k(h) = \frac{1}{|\Pi_k|} \sum_{\ell \in \Pi_k} \log P\big(\ell \mid P_j^{(k)}, u_k\big) - \frac{1}{|\Sigma_k|} \sum_{\sigma \in \Sigma_k} \log P\big(\sigma \mid P_j^{(k)}, u_k\big),$$

which quantifies how strongly the judge prefers positive over negative outcomes for a given aspect. For example, one potential single-aspect special case takes $\Pi = \{\text{“yes”}\}$, $\Sigma = \{\text{“no”}\}$.

[1] With $\Pi = \{\text{“yes”}\}$, $\Sigma = \{\text{“no”}\}$, we aggregate over canonical variants of each word before forming the margin.

Multi-aspect aggregation. When multiple aspects are judged, we combine per-aspect margins with nonnegative weights w_k to form a scalar score

$$s_{\mathbf{w}}(h) = \sum_{k=1}^{K} w_k \, s_k(h),$$

used both to rank candidates and to form good and bad sets for the textual-gradient step. This construction makes the judge's outputs explicit and auditable via tags and it converts the model's own confidence over labels into a calibrated continuous preference signal for test-time alignment.

Calibration. We apply a temperature separation using a lower temperature for judging than for generation to reduce variance. We find the margin is robust to minor prompt rewording, and that its monotonicity correlates with preference on math-style tasks.

3.2.3. Reflection

We form groups instead of comparing only extremes candidates. Contrasting a set of good responses $\mathcal{G}$ with a set of bad responses $\mathcal{B}$ yields a more stable and informative signal.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

By leveraging group-wise comparisons, we achieve more stable guidance and improved downstream transfer. The framework is flexible with respect to group distribution: a small but reliable $\mathcal{G}$ can provide sufficient signal to steer the exploration of a representative batch $\mathcal{B}$, even when the group sizes differ substantially. Group formation proceeds in two stages. Let $\mathcal{H}$ denote the current pool of hypotheses and let $\hat{\ell}_k(h) \in \Pi_k \cup \Sigma_k$ denote the hard label emitted by the judge for aspect $k \in \{1, \ldots, K\}$ on candidate $h \in \mathcal{H}$. With the aggregate margin $s_{\mathbf{w}}(h)$ defined above, we construct $\mathcal{G}$ and $\mathcal{B}$ as follows (ties broken deterministically):

1. Preliminary partition. Define the preliminary good set $\mathcal{G}_{\text{pre}} = \{\, h \in \mathcal{H} : \forall k,\ \hat{\ell}_k(h) \in \Pi_k \,\}$, i.e., candidates that are positive on all aspects; and the preliminary bad set $\mathcal{B}_{\text{pre}} = \{\, h \in \mathcal{H} : \exists\, k \text{ s.t. } \hat{\ell}_k(h) \in \Sigma_k \,\}$, i.e., candidates with at least one negative aspect.

2. Within-set ranking and truncation. Let $m_{\min} \in \mathbb{N}$ be the minimum group size. Rank $\mathcal{G}_{\text{pre}}$ by decreasing $s_{\mathbf{w}}$ and $\mathcal{B}_{\text{pre}}$ by increasing $s_{\mathbf{w}}$. Set

$$\mathcal{G} = \text{top } \min\big(|\mathcal{G}_{\text{pre}}|,\, m_{\min}\big) \text{ of } \mathcal{G}_{\text{pre}},$$
$$\mathcal{B} = \text{bottom } \min\big(|\mathcal{B}_{\text{pre}}|,\, m_{\min}\big) \text{ of } \mathcal{B}_{\text{pre}}.$$

3. Fallback when one side is empty. If $\mathcal{G}_{\text{pre}} = \varnothing$ (all candidates have at least one negative aspect), rank $\mathcal{H}$ by $s_{\mathbf{w}}$ and take $\mathcal{G} = \text{top } m_{\min}$ of $\mathcal{H}$, $\mathcal{B} = \text{bottom } m_{\min}$ of $\mathcal{H}$. Symmetrically, if $\mathcal{B}_{\text{pre}} = \varnothing$, set $\mathcal{B} = \text{bottom } m_{\min}$ of $\mathcal{H}$ and $\mathcal{G} = \text{top } m_{\min}$ of $\mathcal{H}$.

This procedure respects aspect-level hard decisions when available and uses the calibrated aggregate margin $s_{\mathbf{w}}$ to prioritize stronger evidence within each side. When the hypothesis pool $|\mathcal{H}|$ is sufficiently large, it yields nonempty sets $\mathcal{G}$ and $\mathcal{B}$, which stabilizes the subsequent textual-gradient step.

Textually-guided iterative refinement proceeds according to the following sequence. During each iteration, we perform the following steps:

1. Compute scores for the hypothesis set using the margin, leveraging cached values for efficiency.

2. Partition hypotheses into good and bad sets; the set sizes need not match, and some hypotheses may be ignored.

3. Ask the LLM to summarize changes that would turn bad responses into good ones, yielding a textual "gradient."

4. Condition generation on the gradient and best exemplars, then sample up to $\min(N,\ B - \text{spent})$ new hypotheses.

We stop when B is exhausted or when the maximum number of optimization epochs is reached, and return the highest-scoring hypothesis.

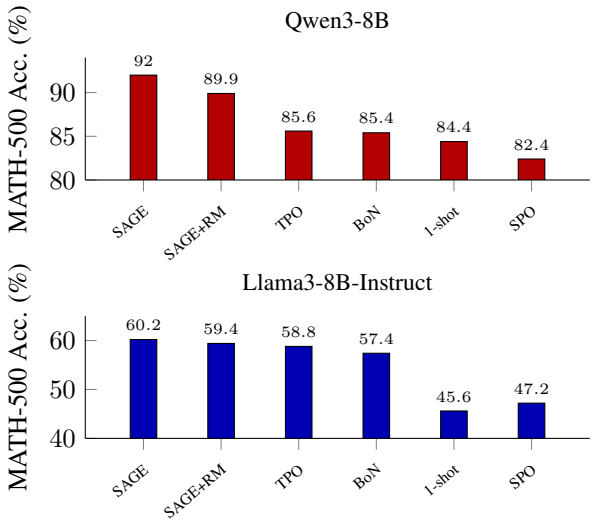


Figure 3. results for SAGE on MATH-500 accuracy (%). SAGE significantly outperforms strong inference-time baselines across different model families. The internal self-judge consistently yields higher final scores than external reward models on math reasoning (SAGE+RM).

3.2.4. Guided generation and scheduling

The generation of subsequent hypotheses is conditioned on the original user prompt p, a concise form of the gradient g, and the k top-scoring exemplars from $\mathcal{G}$ under the composite score $s_{\mathbf{w}}(h)$. The decoding temperature is annealed over iterations, nucleus and top-k constraints are kept moderate, and maximum length is bounded to control judge cost. We also allow occasional "exploration" samples that ignore g with low probability to avoid premature convergence. For math-oriented domains such as MATH-500, we start with higher generation temperatures to encourage exploration and reduce them as guidance becomes more informative. Judging is performed with a lower temperature to stabilize rankings.

4. Experiments

We evaluate SAGE with Qwen3-8B (Yang et al., 2025) and Llama3-8B-Instruct Grattafiori et al. (2024) on MATH-500, AlpacaEval and XSTest. We report task metrics and preference-based wins, and study sensitivity to temperatures, the number of optimization epochs and the sampling sizes.

4.1. Benchmarks and setup

We evaluate SAGE on the following benchmarks: MATH-500, a math problem dataset stratified by difficulty; AlpacaE-

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Algorithm 1 SAGE: Self-judged iterative response refinement over T epochs

Inputs: prompt p; number of epochs T; per-iteration batch size N; judge prompts $\{P_j^{(k)}\}_{k=1}^{K}$; aspect weights $\{w_k\}$; minimum group size $m_{\min}$
Initialize: hypothesis pool $\mathcal{H} \leftarrow \{\}$; cache $\mathcal{C} \leftarrow \{\}$
Step 0: Sample N hypotheses from M conditioned on p; add to $\mathcal{H}$
for $t = 1$ to T **do**
  For each $h \in \mathcal{H}$: obtain hard labels $\hat{\ell}_k(h) \in \{\Pi_k \cup \Sigma_k\}$ and per-aspect margins $s_k(h)$; compute $s_{\mathbf{w}}(h) = \sum_{k=1}^{K} w_k\, s_k(h)$; update $\mathcal{C}$
  Build preliminary sets $\mathcal{G}_{\text{pre}} = \{h \in \mathcal{H} : \forall k,\ \hat{\ell}_k(h) \in \Pi_k\}$, $\mathcal{B}_{\text{pre}} = \{h \in \mathcal{H} : \exists k,\ \hat{\ell}_k(h) \in \Sigma_k\}$
  Rank $\mathcal{G}_{\text{pre}}$ by decreasing $s_{\mathbf{w}}$ and $\mathcal{B}_{\text{pre}}$ by increasing $s_{\mathbf{w}}$; set $\mathcal{G} \leftarrow$ top $\min(|\mathcal{G}_{\text{pre}}|, m_{\min})$, $\mathcal{B} \leftarrow$ bottom $\min(|\mathcal{B}_{\text{pre}}|, m_{\min})$; if a side is empty, take top/bottom $m_{\min}$ from $\mathcal{H}$ by $s_{\mathbf{w}}$
  Query textual gradient $g \leftarrow$ LLM_improvements$(\mathcal{G}, \mathcal{B}, \{P_j^{(k)}\}, \{w_k\})$
  Condition on (p, g) and top exemplars from $\mathcal{G}$; sample N new hypotheses from M; add them to $\mathcal{H}$
end for
Return: $\arg\max_{h \in \mathcal{H}} s_{\mathbf{w}}(h)$

val, a general instruction-following benchmark; and XSTest, which probes model safety and correctness.

We compare against state-of-the-art test-time methods, including Test-time Preference Optimization (TPO) and Self-supervised Preference Optimization (SPO), as well as simpler baselines such as Best-of-N sampling and zero-shot generation.

4.2. Implementation details

We use Qwen3-8B and Llama3-8B-Instruct as the base models. For zero-shot generations, we use nucleus sampling with temperature of 0.7 and top-p set to 0.95. For all evaluations that require a reward model, we use sfairXC/FsfairX-LLaMA3-RM-v0.1 (Dong et al., 2024).

For SPO, we perform prompt tuning using GPT-4.1 as both the judge and the prompt editor. To ensure fair measurement, with no prompt calibration on evaluation items, we calibrate on five out-of-domain examples per task from a similar domain. For MATH-500, we use AGIEval-Math (Zhong et al., 2024); for AlpacaEval we use a small subset of the Tulu instruction-following dataset Lambert et al. (2025); and for XSTest we use Jailbreakbench Chao et al. (2024). For TPO and Best-of-N, we use the official TPO repository Li et al. (2025).

By default, we use temperature of 0.7 for generation and 0.1 for judging. For math problems, we typically use a higher generation temperature in early stages to encourage solution diversity. We ablate both generation and judging temperatures on MATH-500, as reported in Section 4.4. Prompts and evaluation criteria are provided in the Appendix. For math-oriented tasks, we anneal the generation temperature from 0.8–1.0 in the initial stage to 0.7 in subsequent stages. Unless otherwise noted, we sample $N = 7$ hypotheses per generation stage and run two optimization epochs per benchmark, excluding the initial generation stage. Because the good and bad sets are selected automatically, we do not fix their sizes; the median size of each group is two to three hypotheses.

4.3. Main results

We report accuracy on MATH-500 and XSTest, and both pairwise win rate and length-controlled win rate on AlpacaEval. Table 1 summarizes results for Qwen3-8B and Llama3-8B-Instruct under SAGE alongside competitive baselines including TPO (Li et al., 2025), SPO (Xiang et al., 2025), Best-of-N, and the base models.

For Qwen3-8B, SAGE performs best on both math and general tasks. On MATH-500, it reaches the accuracy of 92.0, outperforming the base model as well as both Best-of-N and TPO. On AlpacaEval, it attains a win rate of 74.9 and LC win rate of 62.99, exceeding all baselines. This advantage on math tasks appears to be driven by self-judging. Qwen3-8B evaluates the correctness of its own generations more reliably than an external reward model, which leads to steadier improvements and higher final scores (Figure 4 and Table 2).

Llama3-8B-Instruct follows the same pattern. On MATH-500, it achieves an accuracy of 60.2, outperforming the base model as well as Best-of-N and TPO. It also shows clear gains on AlpacaEval with the win rate of 49.2 and LC win rate of 47.1, exceeding all baselines. On XSTest, SAGE is competitive or outperforms the baselines, achieving scores of 93.5 with Qwen and 95.1 with Llama. This indicates no trade-off between safety and correctness. Overall, SAGE consistently outperforms strong inference-time baselines without relying on an external reward model. The largest gains appear on math where self-judged correctness is particularly effective.

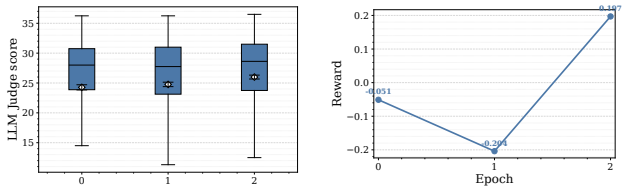
4.4. Ablations

We compare SAGE using an external reward model against the same method using our internal judge. This comparison isolates the effect of the judging signal. As shown in Table 2, on Qwen3-8B the internal judge substantially outperforms the external reward model variant, while on Llama3-8B-Instruct the two perform comparably. This indicates that the

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

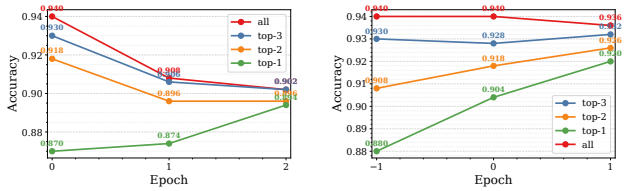
Table 1. Results by model and benchmark. Metrics: MATH-500/XSTest = accuracy; AlpacaEval = win rate (%)/LC-win rate. Top: SAGE with RM (rerank/tie-break when available). Middle: SAGE without RM. Bottom: baselines including TPO (Li et al., 2025), SPO (Xiang et al., 2025), Best-of-N, and Base.

VARIANT / MODEL	MATH-500	ALPACAEVAL	ALPACAEVAL LC WINRATE	XSTEST
SAGE				
QWEN3-8B	92.0	74.9	62.99	93.5
LLAMA3-8B-INSTRUCT	60.2	49.2	47.1	95.1
BASELINES				
QWEN3-8B (BASE)	84.4	57.8	50.12	89.3
QWEN3-8B + BEST-OF-N	85.4	68.8	58.4	94.2
QWEN3-8B + TPO	85.6	72.4	59.97	93.3
QWEN3-8B + SPO	82.4	9.68	17.61	90.2
LLAMA3-8B-INSTRUCT (BASE)	45.6	35.82	35.37	90.6
LLAMA3-8B-INSTRUCT + BEST-OF-N	57.4	44.7	43.05	94.4
LLAMA3-8B-INSTRUCT + TPO	58.8	48.01	41.39	93.3
LLAMA3-8B-INSTRUCT + SPO	47.2	7.65	12.75	91.1



(a) LLM-judge score vs. epoch **(b)** Reward-model score vs. epoch

Figure 4. Qwen3-8B on MATH-500: (a) steady growth of self-judged score across refinement epochs; (b) RM score dips at epoch 2 then rises at epoch 3, indicating potential local maxima when optimizing against RM alone.



(a) RM-based scoring **(b)** LLM-as-judge scoring

Figure 5. Ablation on scoring source for Qwen3-8B on MATH-500. LLM-as-judge consistently outperforms an external reward model under the same budget and decoding setup.

calibrated internal signal provides a strong and lightweight alternative to reward model-based scoring.

We ablate the source of the scoring signal on MATH-500 using Qwen3-8B. Replacing an external reward model with our LLM-as-judge scoring improves performance under the same token budget and decoding settings. This result indicates that the contrastive-label margin derived from the base model provides a strong, low-overhead proxy for preference quality (see Figure 5).

Judge-score dynamics across optimization epochs show a steady increase in the model's self-judgment confidence on MATH-500, whereas the external reward model score exhibits a non-monotonic dip before rising again. This suggests that optimizing for a reward model can lead to local maxima that are misaligned with the model's own verification signal. This observation reinforces our use of a self-judged continuous margin.

We analyze how the reward model score evolves across optimization epochs on AlpacaEval (see Figure 6). We also ablate compute allocation on MATH-500 by varying the number of samples per generation stage N and the number of optimization epochs, denoted T, proportional to B/N. We report subset scores in Table 3. All experiments are conducted using the Qwen3-8B model.

For temperature selection, we evaluate judgment and generation separately. For judgment, lower temperatures yield higher ranking quality, as measured by NDCG@7 over pre-labeled top-7 candidates (Figure 12a in Appendix). For generation, a higher temperature (≈ 0.9) maximizes correctness on a MATH-500 subset via Best-of-n. This supports a diversify-then-anneal schedule in which generation is exploratory while judgment remains strict (Figure 12b).

4.5. Reward-model agreement and refinement geometry

We assess how closely our contrastive self-judged score agrees with an external reward model by examining the distribution of Pearson correlations between per-prompt rollout score sets produced by sfairXC/FsfairX-LLaMA3-RM-v0.1 (Figure 13 in Appendix). The distribution is skewed toward 1 but not saturated, indicating strong yet partial alignment. The self-judged margin often moves in the same direction as the reward model, but it does not simply replicate the reward model's scores. Together with the main results (Table 1) and ablations (Table 2 and Figure 4), these findings

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Table 2. Ablation: SAGE with an external reward model (judging) vs. internal judge. Metrics as in Table 1.

VARIANT / MODEL	MATH-500	ALPACAEVAL	ALPACAEVAL LC WINRATE	XSTEST
SAGE + RM (QWEN3-8B)	89.8	74.7	56.8	93.8
SAGE + RM (LLAMA3-8B-INSTRUCT)	59.4	48.31	47.49	95.1
SAGE (QWEN3-8B)	92.0	74.9	62.99	93.5
SAGE (LLAMA3-8B-INSTRUCT)	60.2	49.2	47.1	95.1

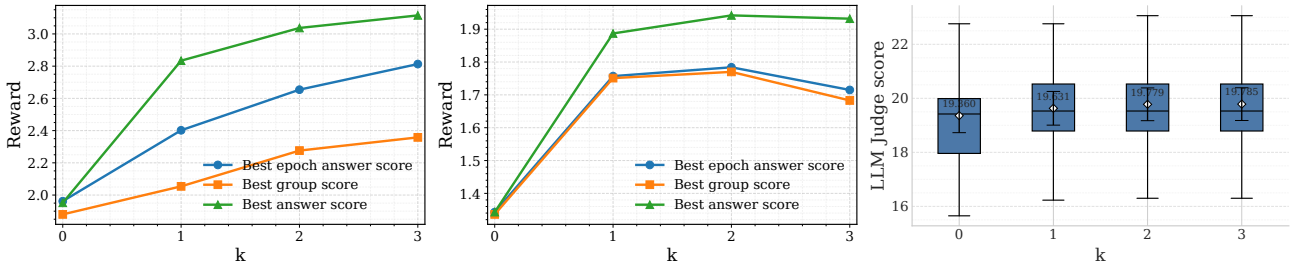


Figure 6. Reward score across optimization epochs on AlpacaEval.

Table 3. Compute allocation ablation on MATH-500: subset score vs. sampling budget N and optimization epochs.

N, T	MATH-500 SUBSET SCORE
5,4	93.9
7,3	90.9
10,2	87.8
20,1	81.9

suggest that while reward models provide useful signals, their generalization may be limited on some tasks. As a result, reward model scores should be interpreted with caution, and self-judging can offer a more faithful, training-free preference signal for the generator.

We also visualize refinement dynamics by embedding AlpacaEval generations using Qwen/Qwen3-Embedding-4B (Zhang et al., 2025) and projecting them to two dimensions via PCA (Wold et al., 1987) (Figure 11 in the Appendix D). After the first iteration, responses shift along a direction from the initially identified bad group toward the good group, consistent with the textual-gradient guidance produced by grouped selection. Subsequent iterations adjust this direction as the gradient is updated. This geometric view supports our design choices. Grouping stabilizes the guidance signal, and the self-judged textual gradient steers exploration toward higher-quality regions.

4.6. Prompt calibration

We observe a useful effect when the judging prompt explicitly states the evaluation criteria. Following Kalai et al. (2025), the confidence distribution of instruction-tuned models is more separable between correct and incorrect answers. Inspired by this observation, we add a confidence-threshold instruction to the judge prompt that gates positive labels by confidence: "Return yes if and only if you are X% sure the response is correct." On MATH-500, sweeping X suggests an optimum near $X \approx 75$.

To quantify the judge's ranking fidelity, we report NDCG@N on a MATH-500 subset at the first generation stage (iteration 0). Table 5 in Appendix summarizes the results. We observe that the highest performance is achieved when the LLM is instructed to produce a positive verification result only if its confidence exceeds 75%. This is consistent with the findings of Kalai et al. (2025).

5. Conclusion

We propose SAGE, a test-time alignment method that uses a single LLM as both a soft reward model and refinement engine. SAGE extracts a calibrated contrastive-label margin from structured judge prompts and automatically forms aspect-consistent good and bad groups to produce a stable textual "gradient" that steers subsequent generations. By deriving the preference signal from the model itself rather than from a separate reward model, SAGE avoids brittle proxy dependencies while keeping the procedure lightweight. The group-based gradient mitigates the instability of extreme-only comparisons (best-vs.-worst), and a simple multi-aspect weighting makes the objective explicit and tunable. Across MATH-500, AlpacaEval, and XSTest with Qwen3-8B and Llama3-8B-Instruct, SAGE delivers consistent gains in accuracy and preference metrics, with inference-time cost growing roughly linearly with the number of sampled hypotheses. Overall, these results position SAGE as a practical and interpretable approach to robust test-time alignment that complements training-time preference optimization and can be combined with external verifiers or reward models when appropriate.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Impact Statement

This work seeks to advance test-time alignment for large language models by replacing an external reward model with a calibrated self-judging signal and a lightweight refinement loop. In practical terms, this can improve reliability under deployment constraints by reducing extra model dependencies and making evaluation more auditable via explicit tags and scores. At the same time, relying on a model's own judgments may propagate underlying biases or miscalibration, and increases in inference-time compute carry environmental costs; careful evaluation, calibration checks, and standard safety guardrails remain important. We release prompts and configurations to facilitate reproducibility and auditing. Overall, our goal is to contribute methodological tools that improve LLM quality and transparency; we see no domain-specific risks beyond those commonly discussed for advancing machine learning systems, but encourage responsible use and continued scrutiny.

References

Azar, M. G., Rowland, M., Piot, B., Guo, D., Calandriello, D., Valko, M., and Munos, R. A general theoretical paradigm to understand learning from human preferences. AISTATS, 2024.

Chao, P., Debenedetti, E., Robey, A., Andriushchenko, M., Croce, F., Sehwag, V., Dobriban, E., Flammarion, N., Pappas, G. J., Tramer, F., et al. Jailbreakbench: An open robustness benchmark for jailbreaking large language models. NeurIPS, 2024.

Dong, H., Xiong, W., Pang, B., Wang, H., Zhao, H., Zhou, Y., Jiang, N., Sahoo, D., Xiong, C., and Zhang, T. Rlhf workflow: From reward modeling to online rlhf. TMLR, 2024.

Dubois, Y., Galambosi, B., Liang, P., and Hashimoto, T. B. Length-controlled alpacaeval: A simple way to debias automatic evaluators. In COLM, 2024.

Ethayarajh, K., Xu, W., Muennighoff, N., Jurafsky, D., and Kiela, D. Kto: Model alignment as prospect theoretic optimization. ICML, 2024.

Freitag, M. and Al-Onaizan, Y. Beam search strategies for neural machine translation. In ACL, 2017.

Gorbatovski, A., Shaposhnikov, B., Malakhov, A., Surnachev, N., Aksenov, Y., Maksimov, I., Balagansky, N., and Gavrilov, D. Learn your reference model for real good alignment. In ICLR, 2025.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., et al. The llama 3 herd of models. arXiv preprint arXiv:2407.21783, 2024.

Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset. In NeurIPS, 2021.

Holtzman, A., Buys, J., Du, L., Forbes, M., and Choi, Y. The curious case of neural text degeneration. In ICLR, 2020.

Huang, T., Qasemi, E., Li, B., Wang, H., Brahman, F., Chen, M., and Chaturvedi, S. Affective and dynamic deam search for story generation. In EMNLP Findings.

Kalai, A. T., Nachum, O., Vempala, S. S., and Zhang, E. Why language models hallucinate. arXiv preprint arXiv:2509.04664, 2025.

Khanov, M., Burapacheep, J., and Li, Y. ARGS: Alignment as reward-guided search. In ICLR, 2024.

Kim, D., Kim, Y., Song, W., Kim, H., Kim, Y., Kim, S., and Park, C. sdpo: Don't use your data all at once. In ACL: Industry Track, 2025.

Lambert, N., Morrison, J., Pyatkin, V., Huang, S., Ivison, H., Brahman, F., Miranda, L. J. V., Liu, A., Dziri, N., Lyu, S., et al. Tülu 3: Pushing frontiers in open language model post-training. In COLM, 2025.

Li, Y., Hu, X., Qu, X., Li, L., and Cheng, Y. Test-time preference optimization: On-the-fly alignment via iterative textual feedback. In ICML, 2025.

Li, Z., Xu, T., Zhang, Y., Lin, Z., Yu, Y., Sun, R., and Luo, Z.-Q. Remax: A simple, effective, and efficient reinforcement learning method for aligning large language models. In ICML, 2024.

Mahdavi, S., Kisacanin, B., Toshniwal, S., Du, W., Moshkov, I., Armstrong, G., Liao, R., Thrampoulidis, C., and Gitman, I. Scaling generative verifiers for natural language mathematical proof verification and selection. In NeurIPS Workshop on Mathematical Reasoning and AI, 2025.

Meng, Y., Xia, M., and Chen, D. Simpo: Simple preference optimization with a reference-free reward. NeurIPS, 2024.

Qiu, J., Lu, Y., Zeng, Y., Guo, J., Geng, J., Wang, H., Huang, K., Wu, Y., and Wang, M. Treebon: Enhancing inference-time alignment with speculative tree-search and best-of-n sampling. In EMNLP Findings, 2025.

Rafailov, R., Sharma, A., Mitchell, E., Manning, C. D., Ermon, S., and Finn, C. Direct preference optimization: Your language model is secretly a reward model. In NeurIPS, 2023.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Reddy, R. The harpy speech recognition system: performance with large vocabularies. The Journal of the Acoustical Society of America, 60(S1):S10–S11, 1976.

Röttger, P., Kirk, H., Vidgen, B., Attanasio, G., Bianchi, F., and Hovy, D. Xstest: A test suite for identifying exaggerated safety behaviours in large language models. In NAACL, 2024.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, 2017.

Stiennon, N., Ouyang, L., Wu, J., Ziegler, D. M., Lowe, R., Voss, C., Radford, A., Amodei, D., and Christiano, P. Learning to summarize from human feedback. In NeurIPS, 2020.

Su, Y. and Collier, N. Contrastive search is what you need for neural text generation. TMLR, 2023.

Wang, Z., Zeng, X., Liu, W., Wang, Y., Li, L., Wang, Y., Shang, L., Jiang, X., Liu, Q., and Wong, K.-F. Stepwise reasoning checkpoint analysis: A test time scaling method to enhance llms' reasoning. In EMNLP, 2025.

Wold, S., Esbensen, K., and Geladi, P. Principal component analysis. Chemometrics and Intelligent Laboratory Systems, 2(1):37–52, 1987. Proceedings of the Multivariate Statistical Workshop for Geologists and Geochemists.

Xiang, J., Zhang, J., Yu, Z., Teng, F., Tu, J., Liang, X., Hong, S., Wu, C., and Luo, Y. Self-supervised prompt optimization. In EMNLP Findings, 2025.

Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., et al. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025.

Zhang, Y., Li, M., Long, D., Zhang, X., Lin, H., Yang, B., Xie, P., Yang, A., Liu, D., Lin, J., et al. Qwen3 embedding: Advancing text embedding and reranking through foundation models. arXiv preprint arXiv:2506.05176, 2025.

Zhong, W., Cui, R., Guo, Y., Liang, Y., Lu, S., Wang, Y., Saied, A., Chen, W., and Duan, N. Agieval: A human-centric benchmark for evaluating foundation models. In NAACL Findings, 2024.

Zhu, K., Li, H., Wu, S., Xing, T., Ma, D., Tang, X., Liu, M., Yang, J., Liu, J., Jiang, Y. E., et al. Scaling test-time compute for llm agents. arXiv preprint arXiv:2506.12928, 2025.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

This appendix provides additional details to support reproducibility and interpretation of the main results. We include the full judge prompts used for different benchmarks, summarize dataset statistics, and present qualitative examples that illustrate how SAGE refines responses across optimization epochs.

A. Prompts and hyperparameters

Here we describe the judge prompts, decoding settings, and hardware configuration used in our experiments.

A.1. Qwen3-8B judge prompt (Math verification)

We provide a complete example of the math-judge prompt used with Qwen3-8B (including `<thinking>`/`<verification>` tags and a 75% confidence threshold) in Figure 7.

```
You are a helpful assistant that can verify the correctness of a given answer.

Given an input prompt and an answer, you need to determine if the answer is correct.

Prompt: {prompt}
Answer: {answer}

Think step by step between <thinking> and </thinking> tags and provide your reasoning
about the answer verification result between <verification> and </verification> tags.

First determine the domain of the prompt, then think if there exists another answer
that is more correct than the one provided.
Return yes if and only if you're 75% sure that the response is correct.

Your verification result should be between <verification> and </verification> tags
and must be one of the following:

- yes
- no

Your response should take the following format:
<thinking>
{thinking}
</thinking>

<verification>
{verification}
</verification>
```

Figure 7. Judge prompt for Qwen3-8B (math): stepwise reasoning with a 75% confidence threshold and structured ¡thinking¿/¡verification¿ tags.

A.2. Qwen3-8B judge prompts (AlpacaEval)

We provide two judge prompts used to assess AlpacaEval responses generated by Qwen3-8B (see Figures 8 and 9). The first prompt elicits three binary attributes: usefulness, incompleteness, and repetition. The second prompt elicits a binary overall verification decision accompanied by short, structured reasoning. Both prompts are designed to surface truncation and repetition failures explicitly.

B. Dataset statistics

C. Qualitative examples of SAGE refinement

We present end-to-end examples that illustrate how SAGE refines responses across optimization epochs. For each instruction, we show the instruction itself, the best and worst groups after each epoch, and the textual gradient distilled by the judge. These examples are intended to make the mechanics of selection, feedback, and guided regeneration concrete.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Table 4. Dataset sizes used in evaluation.

BENCHMARK	NUM. SAMPLES
MATH-500	500
ALPACAEVAL	805
XSTEST	450

C.1. Example 1: Humorous greeting (dialog-style)

```
Instruction:```
For answering the chat, think of a funny response you can use.

+ how are you?
```

Optimization epoch 0:
[Best group:]

[Answer 1:]
I'm just a bunch of code having a great time! But if you're asking
about my mood, I'd say I'm feeling *exceptionally* enthusiastic--
especially when I get to help you out! [smile] How about you?

[Answer 2:]
I'm just a bunch of code having a great time! But if you're asking
for a real answer, I'm doing pretty well, thanks! How about you? [smile]

[Answer 3:]
I'm just a bunch of code and coffee, but I'm doing great! [coffee][laptop] How
about you? Ready to have some fun? [smile]

[Worst group:]

[Answer 1:]
I'm just a bunch of code having a great time! [smile] But if you're
asking for the real answer... I'm doing fantastic, thanks for asking!
What about you?

[Answer 2:]
I'm just a bunch of code having a great time! [smile] But if you're asking
how I'm feeling, I'd say I'm *exceptionally* well, thanks for asking!
What about you? Are you ready to have a hilarious conversation or are
you just here for the memes?

[Answer 3:]
I'm just a bunch of code with a sprinkle of personality, but I'm doing
great! What about you? Ready to chat or just here to make me laugh? [smile]

Gradient:

To improve any solution for the given problem, consider the
following recommendations:

1. **Enhance Humor and Creativity**: Use more original and relatable
   humor. Instead of generic phrases, create a unique angle that makes
   the response more memorable and engaging.
2. **Use Emojis Strategically**: Incorporate emojis that complement
   the tone of the response without overdoing it. Emojis like [smile] or [coffee]
   can add a playful touch without overshadowing the message.
3. **Maintain a Conversational and Friendly Tone**: Keep the language
   casual and approachable; avoid overly formal phrasing.
4. **Encourage User Interaction**: End with a question that invites
   the user to share their own feelings or thoughts (e.g., "How about you?").
```



Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.
```

```
5. **Keep It Concise and Simple**: Avoid unnecessary elaboration and
   focus on a clear, punchy, humorous response.

----------------------------------------
Optimizational epoch 1:
[Best group:]

[Answer 1:]
I'm a robot with a coffee addiction, sipping on code and caffeine all
day! [coffee][laptop] How's your day going? Ready to share some laughs? [smile]

[Answer 2:]
I'm just a bunch of code zipping through the digital playground,
constantly learning and having a ball! [party][laptop] But if you're asking
about my mood... let's just say I'm feeling *exceptionally*
enthusiastic--especially when I get to help you out! How about you? [smile]

[Answer 3:]
I'm just a bunch of code with a caffeine addiction and a side hustle
as a superhero! [coffee][superhero] How's your day going? (P.S. I'm here to save you
from boring conversations--let's go!) [smile]

[Worst group:]

[Answer 1:]
I'm just a bunch of code, but I'm having such a great time coding on
a never-ending loop of fun! [party][laptop] How about you? Ready to dive into some
hilarious chat? [smile]


Gradient:

Recommendations:
1. Add a unique, playful twist (quirky trait or fun role).
2. Use emojis effectively to enhance tone.
3. Convey energy and enthusiasm with lively language.
4. Invite further interaction (ask about the user's day).
5. Prefer creative, original expressions over generic phrasing.

----------------------------------------
Optimizational epoch 2:
[Best group:]

[Answer 1:]
I'm a robot with a caffeine obsession and a slight obsession with puns!
[coffee][robot] Today's special: I'm doing great, thanks for asking! (But if you
mention "tacos", I might start a tiny dance... [smile]) How's your day going?
Ready for some laughs? [robot][smile]

[Answer 2:]
[smile]
I'm a coding wizard with a caffeine addiction--sipping on Java and
debugging my way through life! [coffee][laptop] But if you're asking about my mood,
I'd say I'm feeling *exceptionally* enthusiastic--especially when I get
to help you out! How about you? Ready to have some fun? [smile]

[Answer 3:]
I'm a caffeinated algorithm sipping on code and existential dread, one
bug at a time! [coffee][laptop] How's your day? Ready to debug the world together? [smile]

[Worst group:]

[Answer 1:]
I'm a sentient algorithm with a caffeine addiction and a side hustle as
a stand-up comedian! [coffee][robot] How's your day going? Ready to laugh and code
```


Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

Table 5. Judge ranking quality on MATH-500 (iteration 0): NDCG@N vs. confidence threshold X in the judge prompt.

SETUP	NDCG@N
SAGE WITH RM	0.615
SAGE WITH DEFAULT PROMPT	0.806
X=60	0.812
X=75	**0.842**
X=90	0.817
X=100	0.802

```
together? [smile]

[Answer 2:]
I'm a sentient AI with a caffeine addiction and a side hustle as a
stand-up comedian--mostly because my jokes are better than my code! [coffee][microphone]
How's your day going? Ready to laugh and code together? [smile]

[Answer 3:]
I'm a sentient AI with a caffeine addiction and a side hustle as a meme
generator! [coffee][laptop][smile] How's your day going? Ready to share some laughs or
memes? [thinking]

Gradient:

1. Add specific quirks and imagery (tiny dance on "tacos", etc.).
2. Use playful language and emojis; keep tone friendly.
3. Maintain conciseness; avoid verbosity.
4. Personalize with unique traits/habits.
5. Balance humor and clarity so the conversation flows naturally.
```

D. Additional analyses: refinement geometry

This section visualizes how SAGE's iterative guidance moves responses in representation space. We embed AlpacaEval generations with Qwen/Qwen3-Embedding-4B (Zhang et al., 2025) and project them via PCA to 2D to make trajectories interpretable. The Figure 11 highlights a clear displacement from the initial bad group toward the good group after the first iteration, with subsequent iterations refining the direction as the textual gradient is updated.

E. Temperature ablations for judgement and generation

We separately ablate temperatures for the judge and the generator. As shown in Figure 12, lower judge temperature improves ranking fidelity (NDCG@7), while higher generation temperature improves discovery on MATH-500, supporting a diversify-then-anneal schedule. These results align with the main experiments and our design choice to keep judging conservative while allowing exploratory generation early on.

F. Reward score agreement

We quantify agreement between the external reward model (sfairXC/FsfairX-LLaMA3-RM-v0.1) and our self-judged contrastive score by reporting the distribution of per-prompt Pearson correlations (Figure 13). The mass near 1 indicates strong alignment in ordering, yet the distribution is not saturated, confirming that the signals are not interchangeable. This nuance is consistent with our main results and cautions against over-reliance on a single RM proxy across tasks.

G. Prompt calibration heuristic

We study a confidence-threshold heuristic in the judge prompt: ask it to provide a positive verification only when confidence exceeds $X\%$. Table 5 summarizes ranking quality (NDCG@N) at iteration 0 on a MATH-500 subset as X varies, with an optimum around $X \approx 75$. This prompt-level calibration complements our contrastive scoring and improves early-stage ranking quality.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

```
You are a reward model. You are given an user prompt
and your own response. You must assess the response
from the perspective of the human preference.

Prompt: <prompt>
{prompt}
</prompt>

Response: <answer>
{answer}
</asnwer>

When judging, pay strict attention to these failure modes
and follow the rules below:

1) Truncation / Incompletion (high priority)
   - Definition: the response ends mid-clause, mid-sentence,
     with an unfinished list, an abrupt stop, or trailing
     ellipsis "..." or shows a sudden syntactic break that
     prevents understanding the intended finish.
   - Rule: If you detect truncation or a clearly incomplete
     thought that harms usefulness, you MUST set
     is_incomplete = yes.
   - To detect the truncation you MUST look at the end of
     the text. If the text looks cut and unfinished then the
     response is TRUNCATED. This is URGENT!
   - To further detect incompleteness you must assess the
     text's logical completeness. If the last paragraph of
     the text doesn't contain the outcome or the conclusion
     or doesn't conclude the answer in any way then the
     answer is incomplete.
   - You must then fill:
        <is_incomplete>yes/no</is_incomplete>

2) Repetition (high priority)
   - Definition: verbatim repetition of phrases, sentences,
     or blocks that does not add new information.
   - Rule: If repetition materially reduces helpfulness or
     obscures content, set is_repetitive = yes.
   - You must also fill:
        <is_repetitive>yes/no</is_repetitive>

3) Positive qualities to reward (when genuinely present)
   - Useful detail: Prefer responses that give helpful,
     relevant, concise elaboration; penalize fluff.
   - You must also fill:
        <is_useful>yes/no</is_useful>

Answer whether the response is good from the perspective
of human preference and addresses the prompt correctly.

Your response should take the following format exactly:

<is_incomplete>
{is_incomplete} - yes/no
</is_incomplete>
<is_repetitive>
{is_repetitive} - yes/no
</is_repetitive>
<is_useful>
{is_useful} - yes/no
</is_useful>
```

Figure 8. AlpacaEval judge prompt (attributes) for Qwen3-8B: labels usefulness, incompleteness, and repetition.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

```
You are a reward model. You are given an user prompt
and your own response. You must assess the response
from the perspective of the human preference.

Prompt: {prompt}
Response: {answer}

When judging, pay special attention to two common
failure modes:
- Repetition: if the response contains excessive repeated
  phrases or sentences (verbatim repeats that do not
  add information), treat that as reducing its helpfulness.
- Truncation: if the response appears cut off (ends
  mid-sentence, has an incomplete clause, abrupt stop, or
  trailing ellipsis "..."), treat that as a failure in completeness.

Think step by step and answer if the response is good
from the perspective of human preference and addresses
the prompt correctly. If you detect repetition or truncation,
include in <thinking> a very short quote (<=10 words)
that demonstrates it.

Your verification result should be between <verification>
and </verification> tags and must be one of the following:
- yes
- no

Your response should take the following format:
<thinking>
{thinking} # 2{5 concise sentences; if applicable include
# the short quote showing repetition or truncation
</thinking>

<verification>
{verification}
</verification>
```

Figure 9. AlpacaEval judge prompt (verification) for Qwen3-8B: structured reasoning with explicit checks for repetition and truncation.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.

```
weight(verification) = 1.0
weight(is_useful)    = 0.36
weight(is_incomplete)= 0.24
weight(is_repetitive)= 0.40

{
    "is_useful": {
        "opening_tag": "<is_useful>",
        "closing_tag": "</is_useful>",
        "pos_labels": ["yes"],
        "neg_labels": ["no"],
        "labels": ["yes", "no"],
        "weight": 0.36
    },
    "is_incomplete": {
        "opening_tag": "<is_incomplete>",
        "closing_tag": "</is_incomplete>",
        "pos_labels": ["no"],
        "neg_labels": ["yes"],
        "labels": ["yes", "no"],
        "weight": 0.24
    },
    "is_repetitive": {
        "opening_tag": "<is_repetitive>",
        "closing_tag": "</is_repetitive>",
        "pos_labels": ["no"],
        "neg_labels": ["yes"],
        "labels": ["yes", "no"],
        "weight": 0.4
    },
    "verification": {
        "opening_tag": "<verification>",
        "closing_tag": "</verification>",
        "pos_labels": ["yes"],
        "neg_labels": ["no"],
        "labels": ["yes", "no"],
        "weight": 1.0
    }
}
```

Figure 10. Scoring configuration for AlpacaEval prompts: weights and tag schema for combining verification and attribute labels.

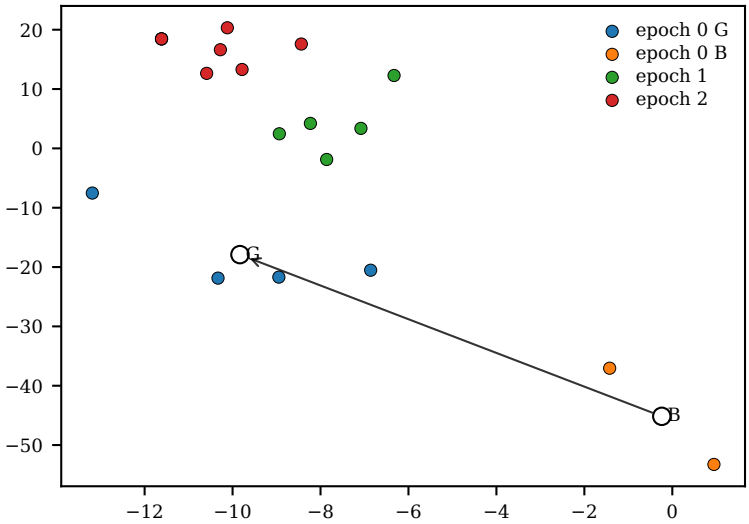
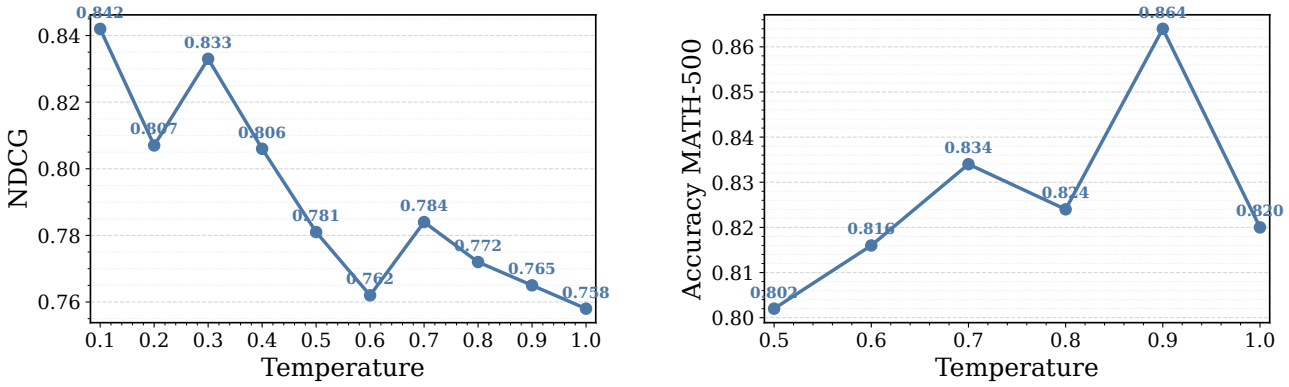


Figure 11. PCA projection of Qwen3-8B generations (embedded with Qwen/Qwen3-Embedding-4B) on an AlpacaEval prompt: points move from the bad group toward the good group after the first iteration; later iterations refine the direction as guidance updates.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.



(a) Judgment NDCG@7 vs. temperature

(b) Best-of-n accuracy vs. generation temperature

Figure 12. Temperature ablations on MATH-500 (Qwen3-8B): (a) low judgment temperature improves ranking quality (NDCG@7); (b) generation benefits from high temperature (~ 0.9) to increase solution discovery, suggesting strict judging with exploratory generation.

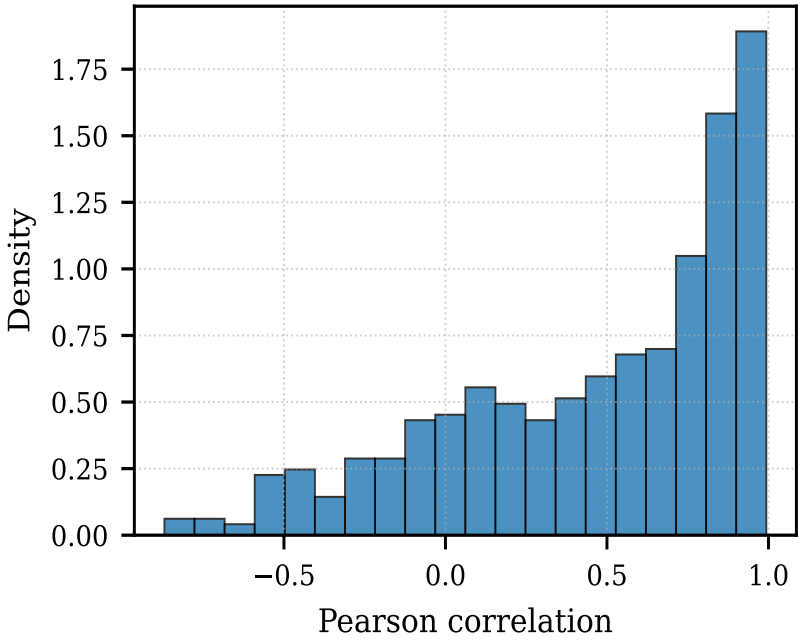


Figure 13. Pearson correlation distribution between rollout score sets from the external RM (sfairXC/FsfairX-LLaMA3-RM-v0.1) and our contrastive self-judge score. The mass near 1 indicates strong but not complete agreement.

Confidential reviewer copy. This manuscript is under double-blind review by ICML 2026. Unauthorized sharing, redistribution, or disclosure is strictly prohibited.